Aikido Security Integration Plan: Hypercar Software Architecture

---

This document details the critical security implementations, configuration workflows, and optional automation features required to secure the embedded software, control logic, telemetry pipelines, and cloud services for the custom hypercar project using Aikido Security.

1. Executive Summary & Core Security Strategy

Modern hypercar engineering relies on high-throughput distributed networks, custom vehicle control units (VCUs), active aerodynamics controllers, and cloud-connected telemetry stacks. Integrating security into the active design and development phase prevents safety-critical vulnerabilities from entering ECU firmware images, DDS bus nodes, and telemetry endpoints.

2. Core Technical Implementations (Required Baseline)

2.1 Repository & Codebase Integration

All active source code repositories must be connected to Aikido to establish automated static analysis and dependency tracking across the software stack.

- **Target Codebases:** C++ and Rust repositories for VCU logic, torque vectoring controllers, active aero modules, Cyclone DDS gateways, and Android Automotive OS / HMI frontend applications.
- **Implementation Steps:**
  1. Navigate to **Assets > Repositories** in the Aikido dashboard.
  2. Connect GitHub/GitLab organizations hosting vehicle control software.
  3. Configure primary default branches (e.g., main, develop) for automated scanning.

2.2 Static Application Security Testing (SAST) & AI Code Analysis

Low-level embedded C++ and Rust code controlling vehicle dynamics requires rigorous memory-safety and logic verification.

- **Scope:** Detect buffer overflows, null pointer dereferences, race conditions, memory leaks, and arithmetic over/underflows in VCU control loops.
- **Verification Check:** Confirm SAST rulesets are set to flag critical severity issues that could trigger hardware failures or controller crashes.

2.3 Open-Source & Dependency Supply Chain Security

Middleware frameworks such as Cyclone DDS, AUTOSAR open layers, and third-party C++ libraries introduce third-party supply chain risks.

- **Scope:** Continuous Software Bill of Materials (SBOM) generation and vulnerability monitoring against CVE databases.
- **Implementation Steps:** Enable automated scanning on manifest files (Cargo.toml, CMakeLists.txt, package.json) to capture compromised upstream packages.

2.4 Access Control & Secrets Management

Prevent API keys, telemetry tokens, and private cryptographic certificates from being embedded into firmware images or public source code.

- **Active Requirement:** Resolve the existing **Access Controls (1)** alert currently flagged in the Aikido dashboard under Issue Types.
- **Implementation Steps:** Rotate affected credentials, move keys to environment variables or dedicated Hardware Security Modules (HSMs), and enable git pre-commit hooks.

3. Feature Implementation Breakdown

Module / Feature	Implementation Stage	Target Architecture / Component	Primary Security Risk Addressed
Repository Linking	Immediate (Required)	All Git Repos (VCU, Aero, HMI)	Unmonitored codebase changes
Access Control Fixes	Immediate (Required)	Cloud Backends & API Gateways	Exposed credentials & service keys
Code Quality Malware Checks	Phase 1 (Required)	Build Pipelines & Package Managers	Malicious supply chain packages & worms
Deep Review on Pull Requests	Phase 1 (Required)	CI/CD Workflows (GitHub Actions)	Flawed VCU logic merged into main branch
Nightly AutoFix PR	Phase 2 (Optional)	Dependency Manifests	Outdated libraries with

Module / Feature	Implementation Stage	Target Architecture / Component	Primary Security Risk Addressed
Creation		& Libraries	known CVEs
Container & Cloud Infrastructure Scanning	Phase 2 (Optional)	Docker Images & Telemetry Servers	Misconfigured cloud containers & APIs
Task Tracker Integration	Phase 2 (Optional)	Jira / GitHub Issues	Siloed security findings outside dev sprint

4. Optional Advanced Features & Platform Configurations

4.1 Code Quality & Malware Readiness

Protecting build environments from targeted supply chain attacks (e.g., malicious npm/crate packages).

- **Configuration:** Under the **Protect** panel, activate **Code Quality Malware Checks** to inspect third-party dependencies for backdoors prior to compilation.
- **Device Protection:** Optional deployment of endpoint or device protection for build hardware and workstation environments.

4.2 AGI Readiness & Automated Code Reviews

Automate deep analysis on every pull request to maintain developer velocity without sacrificing safety controls.

- **Deep Review on PR:** Enable automated LLM-driven deep review on pull requests to catch structural software bugs and subtle logical errors in control routines.
- **Nightly AutoFix PR Creation:** Enable Aikido to automatically generate pull requests for known patchable security vulnerabilities in third-party libraries.

4.3 Automated Task Tracking & Developer Workflows

Streamline security remediation by embedding security tickets directly into existing project management tools.

- **Integration:** Connect task tracking (Jira, GitHub Issues, or Linear) via **Get Started > Set Up a Task Tracker**.
- **Alerting:** Configure custom alert notifications (Slack/Teams/Email) for high and critical issues impacting safety-critical controllers.

5. CI/CD Pipeline Integration Code Example

Below is a sample workflow configuration for integrating Aikido PR checks directly into a GitHub Actions CI/CD pipeline for VCU software build verification:

```
name: VCU Build & Security Gate

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]

jobs:
  security-check:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v4

      - name: Run Aikido Security Scan
        uses: aikido-security/action@v1
        with:
          api-key: ${{ secrets.AIKIDO_API_KEY }}
          fail-on-severity: "high"

      - name: Build VCU Firmware Target
        run: |
          mkdir build && cd build
          cmake ..
          make -j4
```

6. Next Steps & Action Items

1. ---

   Connect all active hypercar repositories under **Assets > Repositories**.
2. Investigate and resolve the single open **Access Controls** issue under **Issue Types**.
3. Enable **Code Quality Malware Checks** and **Deep Review on PR** in the dashboard settings.
4. Link the team's task management system to automate issue assignment during active design sprints.